

Discorso Tesi

Il mio lavoro di tesi è stato caratterizzato dalla progettazione e la realizzazione di un veicolo da trasporto, teleoperato in CANOpen, durante il mio internship in Daikin

Quello che vedremo oggi sarà una breve introduzione sul mio anno trascorso in azienda, lo stato dell'arte che caratterizza i robot mobili con i sistemi di navigazione e i protocolli utilizzati. I requisiti del progetto e i componenti che sono stati selezionati, la proposta di modello cinematico con algoritmo di controllo, l'architettura di comunicazione software sviluppata e cosa è previsto in futuro.

Daikin è un'azienda leader del settore HVAC e la sede Applied di Roma è specializzata nella produzione di grandi macchinari per il settore industriale come UTA e Chiller, con modello di produzione make-to-order.

Durante quest'anno ho lavorato nel reparto di Industrial Innovation che, seppur formato solo due anni fa, ha già iniziato a dare il suo contributo all'azienda con progetti come il MES interno o sistemi di collaudo automatici e, in futuro, robotica mobile per la logistica.

Questo perché attualmente tutta la movimentazione è manuale, senza ottimizzazione degli spazi e decisa sul momento in base alle richieste della produzione.

Quindi, l'obiettivo del mio progetto è stato quello di realizzare un veicolo autonomo che possa essere un prototipo a basso costo per valutare la fattibilità dell'inclusione di progetti di robotica mobile in un ambiente molto dinamico e poco strutturato e, magari, in futuro integrarlo con sistemi di WMS ed ERP per chiudere il ciclo della produzione dalla gestione del magazzino allo stoccaggio post-produzione.

I robot mobili si dividono generalmente in due famiglie. Gli AGV che seguono percorsi prestabiliti e gli AMR, che invece sono dotati di meccanismi in grado di prendere decisioni. Come tipologia di locomozione sono ovviamente comuni quelli su ruote o cingolati, ma negli ultimi anni ci sono molti prodotti dotati di gambe.

In generale, per potersi muovere, questi algoritmi fanno uso di tre sistemi principali:

- dei path planner, per decidere globalmente il percorso da seguire dato un punto iniziale e un punto finale,
- degli algoritmi di obstacle avoidance per aggiustare la traiettoria in modo da evitare ostacoli, anche in movimento,
- E sistemi di localizzazione per stimare la propria posizione e/o mappare l'ambiente intorno al robot.

In particolare per il progetto è stato scelto di usare un algoritmo di SLAM come sistema di

stima della posizione e come algoritmo FAST-LIO2, che sta per Fast Lidar Inertial Odometry. Sfrutta l'unione di un sensore LiDAR e una IMU per stimare la posizione tramite un filtro di kalman esteso e strutture dati per una ricostruzione efficiente della mappa. L'algoritmo è stato testato direttamente sul sensore acquistato, sfruttando l'implementazione fornita dagli autori e configurando in modo opportuno in base al LiDAR usato.

Tra i protocolli di comunicazione usati in ambito industriale è stato scelto il protocollo CAN, che sta per Controller Area Network. Si tratta di un bus seriale tipicamente utilizzato in ambito automotive che garantisce velocità di comunicazioni sufficientemente veloci su cavi di brevi distanze ed offre garanzie sull'affidabilità del canale.

Come protocollo applicativo, la scelta è ricaduta sul protocollo CANOpen, che sfrutta CANBus definendo delle strutture dei pacchetti più complesse per includere funzionalità aggiuntive come una mappa degli indirizzi di memoria dei parametri di un dispositivo CAN o schemi di messaggistica più veloce per l'acquisizione di dati o l'invio di comandi come i PDO. Estensione del protocollo è data dai profili, che sono specifici per il settore di utilizzo. Per esempio, il profilo CiA 402 è quello che dice come deve essere fatta un'implementazione di un nodo CANOpen per azionamenti elettrici, o i profili di controllo: velocità posizione coppia.

Per tradurre il problema di movimentazione in delle specifiche di progetto, è stato scelto un tipo di prodotto da movimentare: un compressore. Oggetto dalla massa importante (quel 730 è uno dei modelli più pesanti) ma dalle dimensioni tutto sommato contenute. Il percorso previsto è quello di portarlo dalla zona di stoccaggio fuori dalla sua area di produzione verso lo stabilimento di produzione dei chiller.

Sovradimensionando, ci siamo posti come target quelli di movimentare 1000 kg tra payload e robot.

Questo ci ha permesso di stilare i seguenti requisiti:

- 1000kg di capacità di carico per il sistema di movimentazione,
- alla velocità massima di 1 m/s
- garantendo autonomia di 4 ore
- sia all'esterno che all'interno,
- con possibilità di manovre in tutte le direzioni, principalmente per parcheggio parallelo,
- ma soprattutto economico rispetto alle offerte sul mercato.

Quindi, calcolando una massa di 1000kg a 1 m/s da raggiungere in 3 secondi, si è stimata una potenza necessaria in piano di 627W e una potenza prevista su una pendenza del 10% di 1 kW e 6.

Per l'alimentazione ci siamo rivolti a FlashBattery, leader del settore in Italia, che ci ha offerto una batteria in linea con le richieste stilate e dotata di BMS che implementa parzialmente CiA 402 per il monitoraggio.

Mentre per la parte di trazione è stata scelta una configurazione con due ruote azionate e

sterzanti e due ruote passive libere. Ci siamo rivolti a CFR, azienda anch'essa italiana che ha verificato il dimensionamento proposto e ha realizzato due motoruote su misura per il nostro caso d'uso. Una motoruota è un sistema dotato di una ruota innestata ad un motore, con un altro motore in verticale per consentirne la rotazione verticale mediante un accoppiamento ad ingranaggi

Per il resto dell'hardware, facciamo una panoramica veloce:

- Computer di bordo con CPU intel Core i5 e 32 GB di ram per alte prestazioni di calcolo, con sistema operativo Linux per l'esecuzione di ROS2
- Lidar Livox Mid 360, con un campo visivo di 360 gradi e una imu a 6 assi integrata,
- encoder e sonde di temperatura su ogni motore,
- I driver sono degli inverter 48V Microphase con supporto all'implementazione CiA 402 per il controllo di motori PMAC
- Adattatore USB per collegare il computer ai driver e alla batteria mediante CAN bus.

Parte centrale della progettazione ha visto poi la modellazione 3D e l'analisi agli elementi finiti di un telaio in acciaio per fare da scheletro al robot. È realizzato con profilati in acciaio 30x30 con travi di rinforzo 20x20. Le dimensioni sono di un metro e ottanta per un metro per un totale di circa 70kg.

Dall'analisi FEM fatta con il compressore preso in esame, si vede che la struttura è in grado di sostenere il carico previsto, con leggeri aumenti delle deformazioni nei punti di scarico dei supporti, ma ampiamente entro i limiti di sicurezza. Si notano poi dei punti in rosso, più critici, ma sono artefatti dovuti a come vengono realizzate le mesh per il calcolo.

Infine, il costo totale per l'hardware ammonta a poco più di 17 mila euro, di circa un ordine di grandezza rispetto ad offerte proposte all'azienda.

Per il modello e il controllo, sono stati analizzati due modelli. Il primo è un modello biciclo, preso dalla letteratura, che si basa sull'approssimazione di un veicolo con quattro ruote sterzanti a uno dotato di due ruote poste frontalmente e posteriormente. L'altro modello è una proposta basata sulla risoluzione di vincoli non-olonomi che non fa approssimazioni geometriche sulla posizione delle ruote.

Entrambi sono modelli planari.

Si vuole quindi controllare il vettore delle coordinate generalizzate q , definito da x , y , e θ , con gli ingressi di controllo che sono le due velocità angolari delle ruote ω_f , ω_r e i due angoli di sterzo δ_f , δ_r .

La posizione delle ruote è a due angoli opposti del veicolo, in $L/2$, $W/2$ e $-L/2$, $-W/2$.

Denotiamo poi con x punto c b e y punto c b la velocità nel sistema di riferimento solidale al robot e assumiamo simmetria geometrica per semplicità, con le distanze tra il centro di massa del robot e gli assi delle ruote pari a $l_f = l_r = L/2$.

Consideriamo angoli positivi in senso anti orario e misuriamo θ come la differenza di orientamento rispetto all'asse x e gli angoli di sterzo rispetto all'asse longitudinale del robot.

Le velocità sono positive quando sono verso la direzione di avanzamento del robot.
Per semplicità la trattazione trascura differenze geometriche dovute all'assemblaggio del robot.

Vediamo il modello a biciclo, caratterizzato dalle seguenti equazioni cinematiche, dove viene introdotto l'angolo beta detto angolo di side-slip, ovvero, la direzione che assume il vettore velocità rispetto alla direzione di avanzamento del robot.

Tale modello ha i vantaggi di essere stato testato in letteratura anche dal punto di vista dinamico ed è stata verificata la controllabilità con controllo LQR sul linearizzato.

Tuttavia, presenta delle singolarità per traslazioni rigide di lato, portando beta a divergere a causa delle tangenti degli angoli di sterzo a ± 90 gradi, proibendo manovre tipo il movimento laterale (detto Crab) o la rotazione su se stessi (detta spin)

Per questo è stato proposto un modello alternativo con le seguenti equazioni, dove possiamo vedere anche una rappresentazione schematica più veritiera del sistema.

Tale modello non presenta singolarità data dagli angoli di sterzo e può gestire tutte le traiettorie di interesse. Tuttavia, non è stata fatta né una modellazione dinamica né tantomeno è stato valutato l'effetto delle ruote passive.

Per quanto riguarda la cinematica inversa: date delle velocità desiderate si vogliono calcolare quegli ingressi di controllo che permettono di raggiungerle.

per il modello a biciclo si è scelto di usare la relazione di compatibilità data dal centro istantaneo di rotazione, che ci dice che la velocità angolare con cui il corpo ruota attorno al centro istantaneo di rotazione deve essere la stessa per entrambe le ruote e pertanto si arriva a scrivere che il prodotto tra il raggio di curvatura posteriore per la velocità angolare anteriore deve essere uguale al prodotto tra il raggio di curvatura anteriore e la velocità angolare posteriore.

Nel caso in cui gli sterzi sono uguali, e quindi le ruote parallele, non esiste un centro di rotazione finito e si eguagliano le velocità.

A questo punto:

- note le velocità possiamo calcolare il valore della velocità desiderata e l'angolo di side-slip che ne consegue.
- Dal modello cinematico mettendo insieme l'equazione di theta punto e beta si può imporre un sistema tale per cui troviamo gli equivalenti angoli di sterzo
- Si calcolano le velocità tramite i raggi di curvatura e la condizione di compatibilità.

Nel caso in cui l'orientamento non cambi, e quindi si procede verso un moto traslatorio dato dall'angolo iniziale, si ha che gli sterzi sono necessariamente paralleli, e si vede che l'angolo di side-slip desiderato è proprio uguale all'angolo di sterzo. A questo punto, possiamo legare l'angolo di sterzo alla velocità desiderata e imporre le velocità angolari come la velocità desiderata diviso il raggio della ruota.

Per la cinematica inversa del modello a doppio sterzo, si vuole risolvere lo stesso problema, però cercando gli sterzi e quelle velocità che soddisfano i vincoli con cui il modello è stato creato. Quindi, gli sterzi sono presi come le arcotangenti delle velocità delle ruote e dalle equazioni di puro rotolamento si ricavano le velocità angolari.

Le equazioni trovate sono state poi testate su quattro traiettorie.

Sono stati usati questi parametri per le grandezze geometriche, e gli attuatori sono stati modellati con un modello del primo ordine con costante di tempo pari a 1 secondo per le trazioni e 2 secondi per gli sterzi e sono state inserite anche saturazioni ai valori massimi raggiungibili dai controlli.

Le traiettorie sono state implementate come dei semplici riferimenti di posizione e velocità per simulare: linea retta, arco di circonferenza, traslazione laterale verticale, rotazione sul posto di 180 gradi.

Lo schema di FF per i test è il seguente:

- riferimento che passa le velocità desiderate
- tramite cinematica inversa si calcolano i riferimenti per gli attuatori che poi forniscono i comandi al modello.
- infine, si integra numericamente con RK4

Qui ho riportato la traiettoria ad arco per semplicità in quanto più significativa e vediamo che né il biciclo né il doppio sterzo asservono la traiettoria, pur mostrando un comportamento simile. Nella tabella sono riportati gli errori quadratici medi di posizione e orientamento e si vede che il biciclo fallisce nelle traiettorie di spin e crab a causa della singolarità.

È stato quindi implementato uno schema di controllo con un PID con FF (spiega schema) In modo riassuntivo, è stato calcolato l'errore in body frame, si calcolano i riferimenti di velocità tramite PID, si ritrasformano in global frame e si procede con la cinematica inversa.

In tabella possiamo vedere i guadagni utilizzati e in particolare:

- il derivatore è implementato come un filtro per ridurre il rumore,
- gli integratori sono saturati per evitare effetto windup
- le velocità sono scalate in modo proporzionale quando dovrebbero superare la velocità massima.

Applicato il controllo e lanciato nuovamente le simulazioni notiamo un miglioramento complessivo rispetto al solo FF, permettendoci di vedere che in generale il modello a doppio sterzo sembra performare meglio in simulazione, oltre che supportare tutte le manovre desiderate.

Parte fondamentale del lavoro ha visto poi la risoluzione del problema di integrazione delle diverse componenti hardware. Questo perché l'adattatore acquistato non è compatibile il

modulo SocketCAN del kernel Linux né con implementazioni CANOpen, ma usa un protocollo proprietario di tipo seriale che incapsula i messaggi can.

(spiega schema)

Per questo è stata proposta e implementata un'architettura software a livelli per integrare in modo trasparente qualsiasi software a livello applicativo come ROS2 o qualsiasi implementazione CANOpen con il sistema. In sostanza, si è costruito un layer di traduzione, chiamato bridge, che fa un inoltro bi-direzionale tra una socket CAN virtuale istanziata nel sistema operativo e la porta seriale USB.

Per la realizzazione è stata quindi scritta una libreria che implementa il protocollo di framing documentato dal produttore che permette di fare encoding e decoding dei messaggi CAN

Configurare la porta seriale e l'adattatore con il rate desiderato e/o filtri per ignorare messaggi. gestire in modo concorrente il bus seriale.

Il bridge usa quindi la libreria in un'architettura a tre thread dove il primo si occupa di inizializzare l'adattatore, la socket CAN virtuale e avviare gli altri due thread, rispettivamente di scrittura e lettura.

(spiega schema)

(spiega grafico finale)

In conclusione: rispetto ai requisiti, sono stati raggiunti i seguenti obiettivi:

Dimensionamento dei componenti

selezione e acquisto materiali

progettazione di un dimostratore con predisposizione per diversi layout di configurazione

proposta un'architettura di controllo per l'inseguimento di traiettorie

sviluppo di un'architettura per integrare ogni sottosistema in modo trasparente usando protocollo CANOpen.

Allo stato attuale, lo sviluppo del software è quasi terminato, a meno dei nodi ros per la generazione delle traiettorie e l'invio dei comandi.

A livello hardware invece tutto l'occorrente è stato acquistato e messo a stock, è stato proposto anche un primo esempio di cablaggio per dei test di integrazione di una singola motoruota

Il piano per lo sviluppo del progetto è

1. completare stack software e integrarlo sul robot completo.
2. Assemblare e cablare il resto dell'hardware
3. validare sperimentalmente il modello misurandone le prestazioni e ottimizzare i parametri di controllo su dati reali.
4. In futuro, estendere il progetto a sistemi multi-robot con integrazione di altri sistemi di gestione aziendali.

5. Includere ulteriori sensori come telecamere RGB-D e scanner di sicurezza.